MLOps PROJECT BRIEF

Project 4

Capstone: End-to-End MLflow Pipeline from Raw Data to Monitored Deployment

One automated pipeline that takes data to a live, monitored model, and fails safely when something goes wrong.

Elective II: Machine Learning Operations (MLOps)

Syllabus topics: 1, 2, 7, 15 (integrating 4, 6, 10, 12, 13)

Prepared by

Dr. Santosh Kumar

1. Project Overview

Scenario

A real-estate analytics start-up predicts asking prices (in ₹ lakhs) for property listings. New batches of listings arrive regularly, and today every retraining and release is done by hand, with no record of which data or code produced which model.

The start-up wants a fully automated pipeline: data in, features built, model trained and validated, registered, deployed and monitored. The pipeline must also stop cleanly, keeping the previous model in service, when data or model quality is not acceptable.

Your role

You are the lead MLOps engineer. You design the system, build it and present it to the founders.

Working mode

Work in pairs.

2. Syllabus Topics Covered

Topic	Syllabus item	Where it appears in this project
1	MLOps fundamentals & lifecycle	Task 1: problem framing and lifecycle map
2	CI/CD pipeline for ML projects	Task 6: automated build and release
7	Monitoring endpoint performance	Task 7: serving and monitoring
15	End-to-end MLOps pipeline	Tasks 3–5: pipeline, gates, tracking and registry
4, 10	Versioning and YAML configuration	Task 2: single-source configuration and versioned data
6, 13	Deployment and orchestration	Tasks 3, 6 and 7
12	Training and tuning	Task 3: training stage

3. Tools & Environment

All tools below are free or open source. AWS services from the original syllabus are replaced as shown.

Purpose	Suggested free / open-source tool	Replaces (AWS)
Tracking & registry	MLflow	SageMaker Experiments / Registry
Orchestration	Prefect or ZenML	Step Functions / SageMaker Pipelines
Versioning	Git + GitHub; DVC	CodeCommit
CI/CD	GitHub Actions; pytest	CodePipeline / CodeDeploy
Packaging	Docker	—
Monitoring	Prometheus + Grafana; Evidently for data drift	CloudWatch / Model Monitor
Configuration	YAML with PyYAML	—

Note: *You may substitute an equivalent free tool if you can justify the choice in your report.*

4. Datasets Provided

The datasets are supplied with this brief. Do not generate or replace them unless a task explicitly says so.

File	Rows	Columns	Description
capstone_house_prices_train.csv	4,000	11	Historical listings with their prices. Use it to build and train the first model.
capstone_incoming_batches.csv	1,500	12	Three later batches of 500 listings each (Batch column = 1, 2, 3), including prices. Use them to simulate new data arriving in production.

Data dictionary (both files)

Column	Type	Description
House_ID	string	Unique listing identifier
Batch	int	Batch number (incoming file only)
Area_sqft	int	Built-up area in square feet
Bedrooms	int	Number of bedrooms
Bathrooms	int	Number of bathrooms
Age_Years	int	Age of the property in years
Distance_to_Center_km	float	Distance to the city centre in km
Neighborhood_Score	float	Neighbourhood quality score from 1 to 10
Has_Parking	int (0/1)	1 = parking available
Property_Condition	category	Poor / Fair / Good / Excellent
City_Zone	category	North / South / East / West / Central
Price_Lakhs	float	TARGET: asking price in ₹ lakhs

Data notes

- Age_Years, Neighborhood_Score and Distance_to_Center_km contain missing values.
- The incoming batches are not guaranteed to resemble the training data. Finding out how they differ is part of the monitoring task.
- House_ID and Batch are not predictive features.
- You may use a dataset from Project 1 or Project 2 instead of the supplied one only with your instructor's written approval.

5. Project Tasks

Each task states the outcome you must achieve and how it will be judged. How you achieve it is your design decision.

Task 1: Problem framing and lifecycle map (Topic 1)

What you must achieve

Define the business objective, the success metrics (an error metric plus a plain-language interpretation for the founders) and the non-functional requirements such as prediction latency and how fresh the model must be.

Draw the system architecture and map it to the MLOps lifecycle.

Acceptance criteria

- A design note of at most two pages with an architecture diagram.
- Success thresholds are numeric and justified.
- Every architecture component is traceable to a lifecycle stage.

Task 2: Configuration, code and data management (Topics 4, 10)

What you must achieve

Make one YAML configuration the single source of truth for the pipeline, and put code, configuration and data under version control so any result can be reproduced.

Acceptance criteria

- A fresh clone reproduces the reported metrics within a stated tolerance.
- No paths, thresholds or hyperparameters are hard-coded.
- Configurations are validated on load.

Task 3: Pipeline design (Topics 13, 15)

What you must achieve

Implement the pipeline as an orchestrated workflow with separate stages: ingest, validate data, build features, train, evaluate, register, deploy and monitor.

Each stage must have defined inputs and outputs and must be re-runnable on its own.

Acceptance criteria

- A diagram of the workflow graph is provided and matches the code.
- Stage-level logs and a run history are available.
- Any single stage can be re-run without repeating the whole pipeline.

Task 4: Validation gates and safe failure (Topic 15)

What you must achieve

Add a data gate (for example schema, value ranges and a limit on missing values) and a model gate (a quality threshold plus a comparison against the current production model).

When a gate fails, the pipeline must stop cleanly with an explanatory message and the previous production model must keep serving.

Acceptance criteria

- At least two failing scenarios are demonstrated: a bad data batch and a model below the threshold.
- The production model and endpoint are unchanged after each failure.
- Gate thresholds are configurable and justified.

Task 5: Tracking, registry and lineage (Topic 15)

What you must achieve

Track every run, register models with stage transitions, and make lineage traceable: from a deployed model you must be able to find the code version, data version, configuration and run that produced it.

Acceptance criteria

- A live lineage demonstration starting from the deployed model.
- Stage transitions in the registry happen only through the pipeline.
- Run records include data version and configuration.

Task 6: Automated build and release (Topic 2)

What you must achieve

A pull request must trigger tests and a lightweight pipeline run. A merge to the main branch must build a deployable image and release the new version. Document how to roll back to the previous version.

Acceptance criteria

- CI history shows a passing and a failing pull request and at least one release.
- The image is tagged so that a running container can be linked to a commit.
- The rollback procedure is written and tested once.

Task 7: Serving and monitoring (Topics 6, 7)

What you must achieve

Serve the production model as a containerised API with operational metrics: latency, throughput, errors and the distribution of predicted prices.

Process the incoming batches to measure data drift and, because prices are supplied, performance drift. Define alert conditions and show which batches trigger them.

Acceptance criteria

- A dashboard or report shows operational metrics and drift over the three batches.
- Alert thresholds are justified, and the batches that trigger them are identified with evidence.
- You explain what the pipeline should do when an alert fires.

Task 8: Final demonstration (Topics 1, 2, 7, 15)

What you must achieve

Present the full loop live or as a recording: a code change, the CI run, the pipeline run, deployment and the monitoring update, followed by one failure scenario.

Acceptance criteria

- The demonstration lasts 5 to 10 minutes.
- Every team member explains a part of the system.
- The failure scenario shows safe behaviour.

6. Deliverables

- One repository with the pipeline, configuration, tests, workflows, Dockerfile and README.
- Design note (maximum two pages) with architecture diagram.
- Workflow graph, run history and registry screenshots.
- Evidence of two failing-gate scenarios.
- Lineage demonstration (recorded or live).
- CI history including at least one release.
- Monitoring dashboard or report covering the three incoming batches.

- A 5–10 minute demonstration.
- A short reflection (one page) on what you would improve with more time.

7. Rules & Constraints

- Use only free tools and free-tier resources.
- Every stage must be automated. Manual copying of files between stages is not allowed.
- Only the pipeline may change the production model.
- Each team member must have commits in the repository; individual contributions are checked.
- No secrets or credentials may be committed to the repository.

8. Academic Integrity

This is a two-person project. Work submitted under your names must be work you and your partner did and can explain.

- Do not copy another group's repository, code, configuration, dashboard, or report, in whole or in part. Two submissions that are substantially the same will both be investigated, regardless of which group worked first.
- During the demonstration, both partners must be able to explain, modify live, and justify every part of the submission without reading from notes you did not write yourselves. An inability to do this is treated as evidence the work is not your own.
- Reusing a prior year's submission, a public GitHub repository, or another course's project as a starting point is not allowed unless you cite it explicitly and your own contribution is clearly separated from it.
- Violations are treated as an academic integrity case under your institute's policy, not simply as a grading deduction.

9. Evaluation Rubric (100 marks)

Criterion	Marks	What is assessed
Problem framing & design	10	Clear objective, metrics and architecture
Pipeline architecture & orchestration	20	Separated, re-runnable stages; matches diagram
Gates & failure handling	15	Both gates work; safe failure; previous model keeps serving
Tracking, registry & lineage	15	Complete records; working lineage demonstration
CI/CD & release	15	PR checks; release on merge; rollback documented
Deployment & monitoring	20	Operational metrics, drift detection, justified alerts
Demonstration & documentation	5	Complete, clear, team participation